

bitfence: a pre-transaction risk layer for autonomous on-chain agents

Dennis Wecker, bitfence

Abstract

Autonomous AI agents execute on-chain transactions without a human in the loop. The risk-assessment tooling in production today was built for a human reader. bitfence is a pre-transaction risk oracle for autonomous agents operating on Solana and Base. It decomposes token risk into four live categories — contract authority, liquidity, holder distribution, and trading — with two categories reserved for future work, and assembles a weighted composite with live-category renormalization, maturity-aware scoring, and circuit breakers surfaced as structured response fields. Scoring runs in a pure, versioned policy engine with an explicit evidence discipline: every fact feeding a circuit breaker is three-valued (known-true, known-false, unknown), breakers fire on known evidence only, and an assessment that could not settle a safety-critical check is delivered with a named degradation and a recommendation capped at human approval. The same assessment contract is exposed through three interfaces: an HTTP/JSON API, a hosted MCP server, and Rust-native bindings for the Rig agent toolkit. Access and settlement happen per call in USDC micropayments on Base, implemented against the Coinbase CDP facilitator. Cold-path latency is a sub-second target; production benchmarking is ongoing. Two of the six categories are reserved injection points rather than shipped work; the confidence value reported with every assessment reflects only the data sources actually consulted.

1. The problem: risk assessment for autonomous on-chain agents

Autonomous agents now execute on-chain transactions without a human in the loop at the moment of execution. Agent frameworks — Rig, ElizaOS, Coinbase AgentKit, MCP tool-calling harnesses — let a language model hold a wallet, read a mandate, and broadcast trades. The operator composes the mandate and reviews the report; the agent is the actor that touches the chain.

Risk-assessment tooling in production today was built for a human reader. Block explorers render transaction histories into scannable tables. Web dashboards surface categorical labels — “honeypot,” “retained ownership,” “high tax” — in panels meant for a trader reviewing an unfamiliar token. Browser wallet warnings intervene when a person is about to confirm a prompt. An agent does not scan, does not review, and does not confirm; it consumes structured fields and routes on them.

The mismatch is interface, not capability. Re-packaging a dashboard behind a REST wrapper does not close it: the response shape (prose plus categorical labels), the billing model (per-account API keys), and the latency tolerance (seconds in a browser tab) all reflect the human audience those tools were designed for.

Risk assessment for an autonomous agent must complete before the transaction broadcasts; an agent cannot unwind a settled transaction. After-the-fact monitoring serves reconciliation and operator telemetry; it does not protect the next trade. The check must run inline, within the latency budget of a single tool call.

A risk layer for autonomous agents must satisfy four requirements:

1. **Interface.** Callable inline as a tool, returning a response shape consistent across chains and integration paths, so a tool-calling model can route on fields rather than interpret prose.
2. **Economics.** Priced per call without account creation or conventional billing.

3. **Interpretability.** A score, an actionable recommendation (PROCEED, REQUIRE_HUMAN_APPROVAL, BLOCK), and structured hard-safety flags surfaced as explicit response fields, not folded into a composite.
 4. **Fail-safe degradation.** A human reviewing a dashboard notices when a panel is blank; an agent consuming a score does not. When an upstream data source is unavailable, the layer must say so explicitly and must not let the outage silently disable the one check that would have caught a scam token.
-

2. Scoring methodology

Risk scores are integers on a 0–100 scale, where 0 is minimal risk and 100 is maximum risk.

The six categories

bitfence decomposes risk into six named categories:

1. **Contract authority (CA).** On-chain ownership and control — mint authority, freeze authority, permanent delegate (Token-2022 for Solana), metadata mutability, upgradeable proxies, SELFDESTRUCT, pause and blacklist selectors.
2. **Liquidity (LQ).** Structural safety of the token’s primary liquidity pool — LP lock or burn status, LP-to-FDV ratio, pool age.
3. **Holder distribution (HD).** Concentration of supply across holders — top-holder share, deployer share, Gini coefficient, total holder count.
4. **Trading (TR).** Behaviours that prevent or penalise selling — honeypot detection, sell and buy taxes.
5. **Cross-chain deployer (XC).** Reserved for cross-chain deployer reputation. Does not currently contribute signals.
6. **ML behavioural (ML).** Reserved for pattern matching against known exploit signatures and on-chain graph anomalies. Does not currently contribute signals.

XC and ML are injection points, not planned categories. The response contract, category weights, and per-category breakdown are in place so a model or a cross-chain dataset can ship without changing the analyzer or the API. No model, training data, or inference pipeline exists today.

Each category emits one or more signals under conventional IDs (CA-01, LQ-02, HD-03, and so on). Within a category, the category score is the maximum of its signal sub-scores: the single worst signal determines the category’s contribution to the composite.

Composite with live-category renormalization

The composite is a weighted average across the six categories. A category is live when its evaluator returns at least one signal. A category that returns no signals — because a data source was unavailable, an adapter call failed, or the category is a reserved stub — drops out of both the numerator and denominator; its nominal weight is redistributed proportionally across the live categories.

Renormalization ensures that an absent category neither pulls the composite toward “safe” nor anchors it to an arbitrary midpoint. The composite reflects only the categories for which real data was obtained. Whether an absent category makes the *assessment* unsafe to act on is a separate question, answered by the evidence discipline below rather than by the score.

Evidence discipline

Every fact that feeds a circuit breaker is three-valued: known-true, known-false, or unknown. A source returning a negative is knowledge — DEXScreener answering “no pairs” is a known absence of a pool. A fetch error is not — it leaves the fact unknown. An unknown is never defaulted into a known, in either direction: missing data can neither fire a breaker nor silently suppress one.

A breaker fires only when every fact it needs is known. When a fact a breaker needs is unknown, the breaker is *indeterminate*, and the assessment is delivered as **degraded**: the response names the unavailable sources explicitly, and the recommendation is capped at `REQUIRE_HUMAN_APPROVAL` — it can never be `PROCEED`. The score itself stays fail-open (dark categories drop out and weights renormalize over live ones), but the recommendation fails safe. An upstream outage cannot cause the system to wave a token through on the strength of the checks it happened to skip.

The cap follows indeterminate breakers, not dark sources. If a source is down but every breaker still reaches a known conclusion from the remaining evidence — for example, authority state is known-clean, so an authority breaker is settled regardless of what the dark source would have said — no check was disabled and no cap applies. Degradation is reported when it exists and enforced when it matters.

Trusted-registry tokens (below) are exempt from the degradation cap: the registry entry is curated evidence covering what the dark source would have proven.

Circuit breakers

Circuit breakers are safety overrides evaluated after the composite. A breaker can raise the score to a forced minimum but cannot lower it: a composite already above a breaker’s floor is never reduced. All breakers are evaluated independently; the final score is the maximum of the applicable floors, and every fired breaker is named in the response.

Every breaker belongs to one of two classes, and the classification is part of the breaker’s definition, not a call-site judgment:

Measured-threat breakers fire on direct measurement of an active threat — the token demonstrably cannot be sold, contains a self-destruct path, or carries a confiscatory measured sell tax. They override everything, including trusted-token status: “a major stablecoin that measurably cannot be sold” means a wrong address or a live compromise, and must block.

1. **Confirmed honeypot** (EVM, via honeypot.is) — sell transactions revert.
2. **SELFDESTRUCT opcode** detected in the contract bytecode.
3. **High sell tax** above a blocking threshold, measured by simulation.

Heuristic breakers fire on risk *inference* — authority configuration, holder concentration, LP lock status. They are strong evidence of danger for unknown tokens, and they floor the score for any token outside the trusted registry. They never override trusted-token status, because the same configuration can be legitimate by design: Circle holds mint and freeze authority on USDC deliberately.

4. **Hidden owner and mint** (Solana) — mint and freeze authority both active on a non-established token.
5. **LP fully unlocked and deployer-held** (both chains) — the deployer can withdraw liquidity instantly.
6. **Extreme holder concentration** — top holders control a disproportionate share.
7. **Deployer concentration on a young pool** — a classic rug-setup pattern.
8. **No DEX pool with very few holders** (EVM) — no discoverable liquidity and thin distribution.
9. **Micro holder count** on a non-mature token.

The final score and recommendation resolve in a fixed precedence order, highest first: measured-threat breakers, then the trusted-token cap, then heuristic breakers, then the degraded-assessment cap, then the composite.

Trigger thresholds and forced-minimum floors are not published. Publishing them would let adversarial deployers engineer tokens that sit immediately below the trigger points. The response tells the calling agent which breakers fired; it does not tell a deployer where the trigger points sit.

Maturity-aware scoring

A naive model penalises long-lived tokens for the same characteristics a new scam token exhibits — retained mint authority, shallow LP-to-FDV ratio, unlocked LP. Three mechanisms address the maturity asymmetry between long-lived and new tokens:

1. **Trusted-token registry** (curated). A single curated registry of established tokens — major stablecoins, wrapped assets, and well-known governance tokens on Solana and Base — with one semantics: a registry token's score is capped in the Low band, it is exempt from heuristic breakers and from the degraded-assessment cap, and it is never exempt from measured-threat breakers. The registry prevents false positives on tokens like Wrapped SOL, where retained mint authority is a design feature rather than a rug setup, without creating a class of token the system will vouch for unconditionally.
2. **Established tokens** (computed, softer test). When the token's primary pool is known to have existed beyond a maturity threshold, authority sub-scores are downgraded, the unlocked-LP penalty is reduced, and the hidden-owner-and-mint breaker is disabled.
3. **Mature tokens** (computed, stricter test). Known evidence of long survival — a pool older than a stricter threshold or a holder base above a large-distribution threshold — exempts a token from the LP-related heuristic breakers.

Computed relief is granted on known evidence only. A token whose pool age cannot be determined is neither established nor unestablished — the fact is unknown, a breaker that needs it goes indeterminate, and the assessment degrades. Missing data never buys an exemption from a threat that known facts established, and never manufactures a threat on its own. Thresholds are not documented, for the same reason breaker trigger points are not.

Confidence

Confidence is a measure of coverage, not a measure of how correct the score is. It reports the fraction of the chain's source-bearing units that fully answered during the assessment.

For each chain, the data sources that exist for that chain are grouped into units; a unit is grounded when every source in it answered. A known negative is an answer: “this token has no DEX pool” grounds the liquidity unit as completely as pool data would. Reserved stubs and sources that do not exist for the chain are excluded from both numerator and denominator — Solana’s confidence is not permanently depressed by the absence of a honeypot source, and an optional source that is not configured reads as narrower coverage by design, not as a failed fetch. A fully grounded assessment reaches 1.0. A transient upstream failure reduces confidence for that request without affecting the system’s nominal coverage.

Confidence is reporting, not enforcement. The safety consequence of missing data is enforced server-side by the degraded-assessment cap described above: a caller does not need to interpret a float correctly to stay safe. Confidence remains useful as a graded signal — an operator can log it, alert on it, or set a stricter policy than the server enforces.

3. Chain coverage and data sources

bitfence supports Solana and Base in production. The choice reflects where agent-driven trading activity currently concentrates.

The Solana adapter queries Helius for on-chain reads — account info, supply, largest accounts, and holder pages — Helius DAS for metadata mutability, DEXScreener for pool reserves and pair creation timestamps, and RugCheck for the LP holder breakdown. The adapter handles both SPL Token and Token-2022 mints; Token-2022 extensions are parsed directly from the TLV stream to detect the permanent delegate extension without taking a dependency on the `spl-token-2022` crate.

The Base adapter queries Alchemy JSON-RPC for bytecode and storage reads, Alchemy’s enhanced `getAssetTransfers` endpoint for deployer detection, DEXScreener for pool and pair data, pair-contract calls for LP burn percentage, honeypot.is for honeypot and sell-tax signals, and, optionally, GoldRush (Covalent) for holder-distribution data. Bytecode scanning uses a PUSH-aware opcode walker to avoid false positives from immediate data bytes matching the `SELFDESTRUCT` opcode.

Holder distribution coverage is asymmetric between chains. On Solana, HD-01 (top-10 concentration) and HD-03 (Gini coefficient) are computable from Helius data on every assessment. On Base, both depend on GoldRush. When a GoldRush key is configured, Base HD coverage matches Solana. When it is not, only HD-02 (deployer share, derived from `getAssetTransfers`) and HD-04 (holder count, estimated as a lower bound from an initial page of ERC-20 transfers) fire; HD-01 and HD-03 do not, and the confidence denominator shrinks accordingly rather than reporting the gap as a failure.

Category	Solana	Base	Notes
Contract authority (CA)	Helius RPC, Helius DAS	Alchemy JSON-RPC	Solana parses Token-2022 TLV directly. Base adds <code>SELFDESTRUCT</code> bytecode scan and EIP-1967 proxy detection.

Category	Solana	Base	Notes
Liquidity (LQ)	DEXScreener, RugCheck	DEXScreener, pair-contract calls	RugCheck provides Solana LP-holder breakdown. On Base, LP burn is computed from pair-contract <code>balanceOf</code> calls against dead and zero addresses.
Holder distribution (HD)	Helius RPC, Helius enhanced	Alchemy enhanced; GoldRush optional	On Base without GoldRush, only HD-02 and HD-04 produce signals.
Trading (TR)	not wired	honeypot.is	Solana has no honeypot data source wired today. TR does not contribute signals for Solana assessments.

Adapters return typed data from a concurrent fan-out; the policy engine consumes the adapters' typed outputs directly, so every fact in an assessment is either grounded in a source that answered or explicitly unknown because one did not. The scoring layer is chain-agnostic: adding a chain is an adapter and signal-wiring change, not a change to the composite, circuit breakers, or response shape.

4. Architecture and integration

bitfence is a single Rust binary exposing one scoring engine through three interfaces: an HTTP/JSON API, an MCP (Model Context Protocol) server, and Rust-native bindings for the Rig agent toolkit. Each interface returns the same `RiskAssessment` structure. Adding a fourth interface means an additional adapter at the edge delegating to the same analyzer; it does not mean a rewrite.

Tool-calling LLM agents — Claude Desktop, MCP clients embedded in IDEs, hosted MCP harnesses — speak MCP natively; a hosted MCP endpoint removes an integration step. Rust-native agents built on Rig bind a typed in-process tool, not an HTTP call. Everything else — scripts, scheduled jobs, other-language frameworks — uses the HTTP API. Standardising the scoring engine rather than the transport is the integration-surface claim.

Internally, a Tower middleware stack handles tracing, CORS, per-IP rate limiting, and body limits. An analyzer function performs validation, cache lookup, per-chain branch, signal computation, and cache write; adapters are the only layer that performs network I/O for chain data. Cold-path latency — when no cache layer hits and adapters fan out to upstream sources — is a sub-second target; production benchmarking is ongoing.

All scoring decisions — composite assembly, evidence grading, breaker evaluation, precedence resolution — live in a pure, versioned policy engine with no I/O of its own. The policy carries an explicit version, and every assessment is stamped with the version that produced it. Because

the policy is pure over typed inputs, the full precedence behaviour — including outage scenarios that are impractical to reproduce against live chains — is exercised in deterministic tests.

A two-tier cache — moka L1 in-process, Redis L2 optional — sits in front of the analyzer. Cache TTLs are keyed to the final risk score rather than to token maturity or adapter source. The rationale is adversarial: if an upstream data source is briefly compromised or manipulated and produces a distorted score, the window during which that score persists in cache is bounded. High-risk false positives are re-evaluated quickly; low-risk scores can safely persist longer. Two further rules keep the cache honest: a cached entry stamped with an older policy version is treated as a miss, so no cached verdict outlives the policy that produced it; and degraded assessments are never written to the shared cache, so an upstream outage is re-checked per request rather than being served as if it were a settled answer. Exact durations are not published.

x402 is implemented against the Coinbase CDP facilitator for USDC micropayments on Base. A request without payment receives a 402 `Payment Required` response whose body carries the x402 challenge plus Bazaar discovery metadata — name, description, JSON-Schema, example input and output — for the Coinbase CDP marketplace. The caller’s client signs a payment and retries the request with a signed payment header; the server verifies the payment via a CDP endpoint with EdDSA JWT bearer authentication, runs the handler, and settles on-chain. If settlement fails, the response is discarded and a 502 is returned; a paid response is never delivered to a non-paying client. A constant-time API-key bypass is available for partner integrations, comparing against a configured key without timing-oracle leaks.

Contextual risk

The `POST /v1/risk/contextual` endpoint layers position-specific analysis on top of the base assessment. Given a token address, a position size in USD, and the caller’s total portfolio size in USD, it returns the base assessment plus a `context` object with the following fields:

1. **Pool liquidity** in the token’s primary DEX pool.
2. **Estimated slippage** for the specified position size.
3. **MEV exposure category** — low, medium, or high — based on position-size-to-pool-liquidity ratio, calibrated to the sandwich-attack patterns in the prior literature.
4. **Portfolio concentration** — the position as a percentage of the caller’s total portfolio.
5. **Safe position size** — the largest position size the endpoint treats as within its safe-slippage band, calculated from the pool’s liquidity depth.
6. **Recommended position cap**, considering both slippage and portfolio concentration.

The endpoint can escalate a recommendation but cannot lower it. A `BLOCK` remains `BLOCK` regardless of position size; a `PROCEED` escalates to `REQUIRE_HUMAN_APPROVAL` when estimated slippage or portfolio concentration cross escalation thresholds.

5. Current scope and extensions

The live system today comprises four active signal categories (CA, LQ, HD, TR) on two chains (Solana, Base) exposed through three interfaces (HTTP, MCP, Rig). The extensions below are research directions, not dated commitments. Each follows from an injection point already present in the code or the API contract.

ML behavioural is a reserved injection point, not a scheduled category. The scoring pipeline holds a typed `evaluate` entry point, a reserved category weight, and reserved signal IDs (ML-01, ML-02); the API response shape accommodates multi-signal ML output without breaking the existing contract. No model, no training data, and no inference pipeline exist today. Work on a supervised classifier over on-chain features and a graph-anomaly detector over transfer sequences is under consideration.

The XC category is reserved for cross-chain deployer reputation and does not currently produce signals. Single-chain deployer detection exists on Base today: `alchemy_getAssetTransfers` identifies a token’s deployer from its first transfer event, and the deployer’s current token and LP balances feed the HD category and the deployer-concentration-on-young-pool circuit breaker. The cross-chain extension — looking up whether a deployer has a history of malicious contracts on other chains — is the natural next step. The data source is an open question: a scan-based approach (expensive, slow, comprehensive), a hosted deployer-reputation feed (faster, vendor-coupled, incomplete), or a hybrid.

TON is the concrete named direction for additional chain support. The adapter pattern generalises: a new chain is an adapter and signal-wiring change, not a scoring rewrite. No ordering or timeline is committed.

The service is listed on Coinbase Bazaar and Clawhub. An ElizaOS plugin is in development.

ethereum/ERCs PR #1638 proposes ERC-8211 (“Smart Batching”), a draft standard for composing multi-step on-chain actions into a single signed intent. If the proposal merges, it becomes a natural upstream point for agents binding multiple on-chain actions, including a pre-transaction risk check, into one signed transaction.

6. Related work and positioning

Three services operate in the adjacent space: GoPlus, RugCheck, and honeypot.is.

GoPlus is a multi-chain token risk API and dashboard that returns categorical risk labels (honeypot, tax, owner privileges, liquidity health, and related fields) to human developers and end-users via REST and a web UI. Its surface shape and account model assume developer integration into human-facing applications.

RugCheck is a Solana-focused token analysis service that surfaces LP holder composition, contract authority state, and community-reported flags through a web UI and public endpoints. The service is single-chain and browser-first, with consumption patterns that follow from that audience.

honeypot.is is a simulation-based EVM service that detects tokens whose sell path reverts, along with estimated buy and sell taxes, returned as a single JSON record per token. The scope is category-specific; bitfence consumes it as an upstream input to TR-01 on Base.

bitfence differs from these along five architectural dimensions:

1. It runs before the transaction rather than after.
2. The same assessment contract is exposed through three interfaces, with MCP and Rig binding agent frameworks natively.
3. Circuit breakers are structured, first-class response fields, not dashboard alerts.

4. It degrades fail-safe: an upstream outage that leaves a safety check unresolved caps the recommendation at human approval and is named in the response, rather than silently weakening the assessment.
 5. Access and settlement happen per call in USDC on Base rather than per account via API keys.
-

References

1. **x402 protocol specification.** Coinbase, `coinbase/x402` repository (<https://github.com/coinbase/x402>); CDP facilitator documentation (<https://docs.cdp.coinbase.com/x402/>)
2. **Model Context Protocol specification.** Anthropic. <https://spec.modelcontextprotocol.io/>
3. **ERC-8211: Smart Batching.** `ethereum/ERCs` pull request #1638. Draft, pending merge. <https://github.com/ethereum/ERCs/pull/1638>
4. **Xia et al.** “Trade or Trick? Detecting and Characterizing Scam Tokens on Uniswap Decentralized Exchange.” *ACM Internet Measurement Conference (IMC)*, 2021.
5. **Mazorra, Adan, Daza.** “Do Not Rug on Me: Leveraging Machine Learning Techniques for Automated Scam Detection.” *Mathematics*, vol. 10, no. 6, 2022.
6. **Cernera et al.** “Token Spammers, Rug Pulls, and Sniper Bots: An Analysis of the Ecosystem of Tokens in Ethereum and in the Binance Smart Chain (BNB).” *USENIX Security Symposium*, 2023.
7. **Daian et al.** “Flash Boys 2.0: Frontrunning, Transaction Reordering, and Consensus Instability in Decentralized Exchanges.” *IEEE Symposium on Security and Privacy*, 2020.
8. **Helius.** Solana RPC, Digital Asset Standard (DAS), enhanced token-account API. <https://helius.xyz>
9. **Alchemy.** Base JSON-RPC and enhanced `getAssetTransfers`. <https://alchemy.com>
10. **GoldRush (Covalent).** `token_holders_v2` endpoint. <https://goldrush.dev>
11. **DEXScreener.** Pool reserves, FDV, pair creation timestamps. <https://dexscreener.com>
12. **RugCheck.** Solana LP holder breakdown. <https://rugcheck.xyz>
13. **honeypot.is.** EVM honeypot and sell/buy tax detection. <https://honeypot.is>